UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA

Engineering, Built Environment and IT
Department of Computer Science

Imperative Programming
COS 132

Semester test 2

23 May 2024 (17:30 to 19:00)

Name (Initials and Surname): [redacted]

Student number: [redacted]

Degree programme (BSc CS/IKS/Other, BEng, BISMM, Other (specify) [redacted]

Examiners

Mr Sheanesu Makura, Ms Seani Rananga, Mr Tlou Lebelo, Ms Inge Odendaal, Mr Mark Botros and Dr Linda Marshall

Instructions

1. **Write your name and student number on the paper in the space provided.**
2. Read the question paper carefully and answer all the questions.
3. Write all your answer on the question paper. All answers must be written in indelible pen. Pencil may not be queried.
4. The assessment opportunity comprises of **6** questions on **11** pages. Refer to the table below for the marks per question.
5. This is a closed book assessment.
6. One and a half (**1.5**) hours have been allocated for you to complete this paper.
7. In the event of loadshedding, extra time will be given depending on how long it takes the generators to "kick in".

Question:	1	2	3	4	5	6	Total
Marks:	5	11	20	9	15	11	71
Score:	4		20	9	5	4,5	

Excellent

Full marks is: 60

1. For each of the following multiple choice questions, write the most correct answer from the options given to the left of the options.

(a) Which of the following is a good sentinel value for a program that inputs the number of hours each employee worked during the week? (1)

C A. -9
B. 32
C. 45.5
D. 7
E. none of the above

(b) Which of the following updates the total accumulator variable by the value in the sales variable? (1)

D A. total = total + sales;
B. total = sales + total;
C. total += sales
D. all of the above
E. none of the above

(c) What is the output of the following code segment? (1)

```
int count = 1;
do {
     count = count + 1;
     cout<< count<< " ";
} while (count <= 4);
```

D A. 1 2 3 4 5
B. 1 2 3 4
C. 2 3 4
D. 2 3 4 5
E. none of the above

(d) Given below are three implementations of a function to increment a variable by 1: (1)

i

```
void increment(int a) {
        a = a + 1;
}

int main() {
        int num = 5;
        increment(num);
}
```

ii

```
void increment(int &a) {
        a = a + 1;
}

int main() {
        int num = 5;
        increment(num);
}
```

iii

```
void increment(int *a) {
        *a = *a + 1;
}

int main() {
        int num = 5;
        increment(&num);
}
```

Which of these implementations would successfully increment the variable num by 1?

E A. i only
B. ii only
C. iii only
D. i and ii only
E. ii and iii only

(e) Study the following code and answer the question that follows: (1)

```
void changeValue(int *ptr) {
    *ptr += 5;
}

int main() {
    int num = 5;
    int *ptr = &num;
    changeValue(ptr);
```

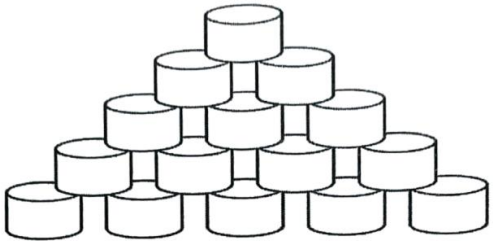
```
    std::cout << num << std::endl;
    return 0;
}
```

When the program is run, what is the output?

A. 5
B. 10
C. 15
D. Compiler Error
E. The address of num

2. For each of the following scenarios, write or complete the functions requested.

(a) Students want to build a tower with cans as indicated in the example below. (5)



If the number of cans in the bottom row is 5, the students will need 15 cans. Each row contains one less can than the previous row. $(5 + 4 + 3 + 2 + 1 = 15)$. The number of cans in the bottom row may vary. Write a function, called `calculateCans`, that returns the total number of cans required to build a tower, if the number of cans for the bottom row is provided as a parameter to the function.

```
int calculateCans(int bottom) {
    int numCans = 0;
    for (int i = bottom; i > 0; i--) {
        numCans += i;
    }
    return numCans;
}
```

(b) **Write a function in C++ called `printX` that prints an "X" shape made up of asterisks (*). The function should take a single integer parameter named size, which determines the `height` and `width` of the "X". The size must be an odd number. If size is not odd, the function should return an error message stating "Size must be an odd number.".** (6)

For example if `size = 5`, the function should print:

```
*   *
 * *
  *
 * *
*   *
```

Complete the function by filling in the missing parts of the code below:

```
#include <iostream>
using namespace std;

void printX(int size) {
    if ( ---(a)--- ) {
        cout << "Size must be an odd number." << endl;
        return;
    }
```

```
    for (int i = 0; ---(b)--- ; i++) {
        for (int j = ---(c)--- ; ---(d)--- ; ---(e)---) {
            if (i == j || ---(f)--- ) {
                cout << '*';
            } else {
                cout << ' ';
            }
        }
        cout << endl;
    }
}
```

(a) - size % 2 == 0

(b) - i <= size

(c) - 0

(d) - j < size

(e) - j ++

(f) - j % 2 == 0

3. Study the C++ code for each of the listings that are given and answer the questions that follow the respective listings. You my assume the required libraries have been correctly included and any required variables correctly declared and initialised.

(a)

```
cout << "Enter a number: ";
cin >> num;
if (num > 0) {
        cout << num << " is positive." << endl;
        if (num % 2 == 0) {
            cout << num << " is even." << endl;
            num *= 2;
        } else {
            cout << num << " is odd." << endl;
            num *= 2;
        }
} else {
        cout << num << " is negative." << endl;
        if (num % 2 == 0) {
            cout << num << " is even." << endl;
            num *= 2;
        } else {
            cout << num << " is odd." << endl;
            num *= 2;
        }
}
```

i. Based on the code given above, test it using 6 and -5 as input values. Write the output. (4)

using 6 !
6 is positive
6 is even.

using -5 !
-5 is negative.
-5 is odd.

4

ii. Briefly explain the purpose of factorisation in C++? (2)

Factorisation is used to remove repeated code and use up less memory.

iii. Apply bottom factorisation to the above code provided, such that code equivalence is achieved. (4)

```
cout << "Enter a number: " << endl;
cin >> num;
if (num > 0) {
    cout << num << " is positive." << endl;
    if (num % 2 == 0) {
        cout << num << " is even." << endl;
    } else {
        cout << num << " is odd." << endl;
    }
} else {
    cout << num << " is negative." << endl;
    if (num % 2 == 0)
```

Answer on extra page at the back.

OK.

(b)

```
1  #include <iostream>
2  using namespace std;
3
4  void myfunc(int num, int* myPtr) {
5      *myPtr = 0;
6      while (num != 0) {.
7          int tt = num % 10;
8          *myPtr = *myPtr * 10 + tt;
9          num /= 10;
10     }
11 }
12
13 int main() {
14     int num;
15     cout << "Enter a number greater than 9: ";
16     cin >> num;
17     int newNum;
18     myfunc(num, &newNum);
19     cout <<newNum << endl;
20
21     return 0;
22 }
```

i. Explain the purpose of the myfunc() function in line 4. (2)

~~To extract the digits of~~

To reverse the digits of the passed in num value.

ii. What is happening in line 18? (2)

The ~~for~~ void function ~~myfunc~~ myfunc() is called with the input of the user and another variable passed by reference. The user input gets reversed and stored in newNum.

iii. Convert the while-loop in lines 6 to 10 to a for-loop. If a line of code does not change, simply write // same as original line N, where N is the line number in the original code. (4)

```
for (num; num != 0; num != 10) {
    // same as original line 7
    // same as original line 8
}
```

good

iv. Write the output in the case a user enters 48 for input. (2)

output: 84

4. The program below implements the Ackermann function. The function uses cout statements to trace its execution at different recursion depths. Examine the code and write down the exact output that this program will produce when executed (9)

```
int ackermann(int m, int n, int depth) {
    cout << "ack(" << m << "," << n << ") at depth " << depth << endl;
    if (m == 0) {
        cout << "Returning " << n+1 << " from ack(" << m << "," << n << ")
            at depth " << depth << endl;
        return n + 1;
    } else if (n == 0) {
        int result = ackermann(m - 1, 1, depth+1);
        cout << "Returning " << result << " from ack(" << m << "," << n <<
            ") at depth " << depth << endl;
        return result;
    } else {
        int result = ackermann(m, n - 1, depth+1);
        result = ackermann(m - 1, result, depth+1);
        cout << "Returning " << result << " from ack(" << m << "," << n <<
            ") at depth " << depth << endl;
        return result;
    }
}

int main() {
    int m = 1;
    int n = 1;
    cout << ackermann(m, n, 1);
    return 0;
}
```

Output:

ack(1, 1) at depth 1
ack(1, 0) at depth 2
ack(0, 1) at depth 3
Returning 2 from ack(0, 1) at depth 3
Returning 2 from ack(1, 0) at depth 2
ack(0, 2) at depth 2
Returning 3 from ack(0, 2) at depth 2
Returning 3 from ack(1, 1) at depth 1
3

5. **You have been asked to help encrypt and decrypt text messages so that your rivals are unable to steal your companies trade secrets. As you are a beginner coder, a simple Ceasar cypher will need to be implemented.**

(a) Write the encode function for the Ceasar cypher. The function is defined as follows: (5)

```
string encode(string, int);
```

The function accepts a string and an offset as parameters and returns a new string after it has been encoded. Each character in the string is changed based on the offset given. The letter 'H' with offset 2, will be changed to 'J'. Make use of the indexes of the string to write your function. That is, if the string parameter is referred to as `str`, `str[0]` will be the first character. You may make use of the `length` function defined in the string class to determine the length of the string.

```
string encode (string, int){
~~for(int i=0; i<5~~
string str = string;
int offset = int;
for(int i=0; i<=str.length(); i++){
  str[i] = str[i] + offset;
}
return str;
}
```

3

(b) It does not help sending encrypted messages into the *ether* if you cannot decrypt them. Write the decrypt function. It receives a reference to a string as a parameter along with the same offset value as what the string was encoded with. Make use of only pointer arithmetic and write the decrypt function with the following definition. (5)

```
void decode (string &,int);
```

```
void decode (string &, int) {
  string *pntr = &string;
  for (int i=0; i<=
```

(c) Is it possible to call encode as follows, `string str = encode("Hello word",5);`? Provide reasons for your answer. (2)

Yes it is possible. The string literal "Hello word" will be implicitly converted to and assigned to the str variable.

(d) Show how you would call the decode function. Provide a short explanation as to why this is the case. (3)

decode ("Helloworld", 6)

The string literal must be passed by reference to fit the parameter list.

6. Given the following code snippet, answer the following questions.

```
string z = "";
for ( int y=19, x=10; y>10 ; y--, x=10 ) {
        do {
                z += (x>16)? "A": (x>14)? "B": (x>12)?"C":"D" ;
                z += "\n";
        } while(++x < y);
}
```

(a) Perform the following translations.

i. Ternary-if to a switch-statement. (4)

```
if (x >16) {
  z += "A";
} else if (x > 14) {
  z += "B";
} else if (x >12) {
  z += "C";
} else {
  z += "D";
}
```

(3)

ii. For-loop to a while-loop.

```
int y = 19;
~~while(y>10){~~
int x = 10;
while(y>10){
  do{
    z+= (x>16)? "A" : (x>14)? "B" : (x>12)? "C" : "D";
    z+= "\n";
  }while(++x<y); y-=1; x=10; }
```

3

iii. Do-while to a for-loop. (2)

```
~~for(;;)~~
for(;++x<y;0){
    z+=(x>16)? "A" : (x>14)? "B" : (x>12)? "C" : "D";
    z+= "\n";
}
```

1,5

(b) How many iterations/repetitions will be made, that is how many times will the ternary-if statements be evaluated? (2)

The do-while loop will never end.

The ternary if statements will be evaluated infinitely.

0

Make use of this page for all your rough work or answers for which you could not complete in the space provided. Make sure you clearly mark the questions and refer the marker to where you have included the answer.

```
Using 6:          Using -5:
6 is positive     -5 is negative
6 is even         -5 is odd
```

Q3.)(a)(iii)

```
cout << "Enter a number: " << endl;
cin >> num;
if (num > 0) {
  cout << num << " is positive." << endl;
} else {
  cout << num << " is negative." << endl;
}
if (num % 2 == 0) {
  cout << num << " is even." << endl;
} else {
  cout << num << " is odd." << endl;
}
num *= 2;
```

good.

```
ack(1, 1) at depth 1
ack(1, 0) at depth 2
ack(0, 1) at depth 3
Returning 2 from ack(0, 1) at depth 3
Returning 2 from ack(1, 0) at depth 2
ack(0, 2) at depth 2
Returning 3 from ack(0, 2) at depth 2
Returning 3 from ack(1, 1) at depth 1
3
```